

Ray Tracing Assignment - CSE410

Project Structure

```
project/
├── 2005XXX_main.cpp ..... # Main file with OpenGL setup and camera controls
├── 2005XXX_classes.h      # Header file with all class definitions
├── intersection_methods.cpp # Implementation of ray-object intersection methods
├── bitmap_image.hpp ..... # Bitmap image handling for output
├── scene.txt              # Main scene file (use the provided one)
├── scene_test.txt .....  # Simple test scene
├── Makefile .....        # Compilation script
└── README.md             # This file
```

Compilation Instructions

Linux/macOS:

```
bash

# Install dependencies (Ubuntu/Debian)
sudo apt-get install freeglut3-dev libglu1-mesa-dev

# Compile
make

# Or compile manually
g++ -std=c++11 -Wall -O2 -o raytracer 2005XXX_main.cpp -lGL -lGLU -lglut -lm
```

Windows (MinGW):

```
bash

g++ -std=c++11 -Wall -O2 -o raytracer.exe 2005XXX_main.cpp -lopengl32 -lglu32 -lfreeglut -lm
```

Running the Program

1. Make sure you have a `scene.txt` file in the same directory as the executable
2. Run the program:

bash

`./raytracer`

Controls

Camera Movement:

- **Arrow Keys:**
 - ↑: Move forward
 - ↓: Move backward
 - ←: Move left
 - →: Move right
- **Page Up/Down:**
 - Page Up: Move up
 - Page Down: Move down

Camera Rotation:

- **Number Keys:**
 - 1: Rotate/Look left
 - 2: Rotate/Look right
 - 3: Look up
 - 4: Look down
 - 5: Tilt clockwise
 - 6: Tilt counterclockwise

Ray Tracing:

- **0:** Capture current view as bitmap image

Scene File Format

The scene.txt file should follow this format:

[recursion_level]

[image_width]

[number_of_objects]

[object_definitions...]

[number_of_point_lights]

[point_light_definitions...]

[number_of_spot_lights]

[spot_light_definitions...]

Object Types:

1. Sphere:

sphere

[center_x] [center_y] [center_z]

[radius]

[color_r] [color_g] [color_b]

[ambient] [diffuse] [specular] [reflection]

[shininess]

2. Triangle:

triangle

[x1] [y1] [z1]

[x2] [y2] [z2]

[x3] [y3] [z3]

[color_r] [color_g] [color_b]

[ambient] [diffuse] [specular] [reflection]

[shininess]

3. General Quadric Surface:

general

[A] [B] [C] [D] [E] [F] [G] [H] [I] [J]

[ref_x] [ref_y] [ref_z] [length] [width] [height]

[color_r] [color_g] [color_b]

[ambient] [diffuse] [specular] [reflection]

[shininess]

Light Types:

1. Point Light:

```
[pos_x] [pos_y] [pos_z]  
[color_r] [color_g] [color_b]
```

2. Spot Light:

```
[pos_x] [pos_y] [pos_z]  
[color_r] [color_g] [color_b]  
[dir_x] [dir_y] [dir_z]  
[cutoff_angle]
```

Features Implemented

✓ Core Features:

- Fully controllable camera with 6DOF movement
- Ray-sphere intersection
- Ray-triangle intersection
- Ray-general quadric surface intersection
- Checkerboard floor
- Phong lighting model (ambient, diffuse, specular)
- Point lights and spot lights
- Shadow calculation
- Recursive reflection
- Bitmap image output

✓ Advanced Features:

- Multiple recursion levels
- Multiple light sources
- Proper normal calculation for all surfaces
- Bounding box clipping for quadric surfaces

Output

- Images are saved as `Output_1.bmp`, `Output_2.bmp`, etc.
- Each time you press '0', a new image is captured with an incremented counter

Testing

Start with the provided `scene_test.txt` file for initial testing:

1. Copy `scene_test.txt` to `scene.txt`
2. Run the program
3. Use camera controls to navigate
4. Press '0' to capture images
5. Verify the output images are generated correctly

Troubleshooting

1. **Compilation errors:**
 - Make sure OpenGL/GLUT libraries are installed
 - Check that all header files are in the same directory
2. **Black images:**
 - Check scene file format
 - Verify camera position and orientation
 - Ensure objects are within view
3. **No shadows/reflections:**
 - Verify lighting parameters in scene file
 - Check recursion level settings
 - Ensure reflection coefficients are > 0

Performance Notes

- Larger image sizes will take longer to render
- Higher recursion levels increase rendering time exponentially
- Complex scenes with many objects will be slower
- Progress is displayed during image capture